Validation Agent

(React + Python FastAPI)

This repository contains a full-stack Validation Agent application that checks whether a user-submitted JSON object conforms to a predefined schema and business rules.

Project Structure

```
validation-agent/
  backend/
      main.py              # FastAPI application and validation logic
      requirements.txt     # Python dependencies
      payloads.json        # Local storage for saved payloads
      logs.jsonl           # Validation logs (auto-generated)
  frontend/
      src/
          App.jsx          # Main React component
          App.css          # Application styling
          App.test.jsx     # Vitest unit tests
          setupTests.js    # Test environment setup
      package.json          # Node dependencies and scripts
      vite.config.js        # Vite bundler configuration
```

How to Run Locally

1. Backend (Python)

1. Navigate to the `backend` directory:

   ```
   cd backend
   ```

2. Create and activate a virtual environment (recommended):

   ```
   # Windows
   python -m venv venv
   venv\Scripts\activate

   # Mac/Linux
   python3 -m venv venv
   source venv/bin/activate
   ```

3. Install the dependencies:

   ```
   pip install -r requirements.txt
   ```

4. Start the FastAPI server:

   ```
   uvicorn main:app --reload
   ```

The backend will be available at *http://127.0.0.1:8000*.

2. Frontend (React)

1. Open a new terminal and navigate to the `frontend` directory:

   ```
   cd frontend
   ```

2. Install the dependencies:

   ```
   npm install
   ```

3. Start the development server:

   ```
   npm start
   ```

The UI will automatically open in your browser.

How to Run Tests

The frontend includes 4 robust unit tests using Vitest and React Testing Library that mock the API client to ensure the UI behaves correctly under different scenarios.

To run the tests, navigate to the `frontend` directory and run:

```
npm test
```

Assumptions & Design Decisions

- **Local Storage over Database**: As per the requirements, no external databases were used. Payloads are persisted to a local `payloads.json` file on the backend, ensuring a lightweight and easy-to-run setup.
- **Latency Simulation**: A random delay between 150ms and 500ms is injected into the validation endpoint using `time.sleep()` to simulate real-world processing times and allow the frontend's "Validating..." loading state to be observable.
- **Mocking for Tests**: The global `fetch` API is mocked in the Vitest suite using `vi.fn()`. This ensures the UI tests are completely isolated from the backend, executing instantly without requiring the Python server to be running.
- **UI/UX**: The frontend was built with standard CSS Flexbox and variables to create a clean, modern aesthetic without introducing heavy component libraries. A two-column layout ensures the validation form and saved payloads are simultaneously accessible on desktop displays.